



*Type Script

Ahmed Elashry
aelashry@outlook.com

*Typescript is nothing but JavaScript
with some additional features

- *Declared using **interface** keyword
- *TS compiler shows error when Interface signature and **implementation does not match**
- ***Optional properties** can be declared for an interface (**using ?**)

```
interface IEmployee {  
    empCode: number;  
    empName: string;  
    getSalary: (number) => number; // arrow function  
    getManagerName(number): string;  
}
```

***Interface**

*Typescript provides a way to mark a property as read only. This means that once a property is assigned a value, it **cannot be changed!**

Example: Readonly Property

```
interface Citizen {  
    name: string;  
    readonly SSN: number;  
}  
  
let personObj: Citizen = { SSN: 110555444, name: 'James Bond' }  
  
personObj.name = 'Steve Smith'; // OK  
personObj.SSN = '333666888'; // Compiler Error
```

***READ ONLY PROPERTIES**

- * Interfaces can extend one or more interfaces. This makes writing interfaces flexible and reusable.

Example: Extend Interface

```
interface IPerson {  
    name: string;  
    gender: string;  
}  
  
interface IEmployee extends IPerson {  
    empCode: number;  
}  
  
let empObj: IEmployee = {  
    empCode: 1,  
    name: "Bill",  
    gender: "Male"  
}
```

* EXTENDING INTERFACES

- * Similar to languages like java and C#, interfaces in typescript can be implemented with a class. The class implementing the interface needs to strictly conform to the structure of the interface

Example: Interface Implementation

```
interface IEmployee {  
    empCode: number;  
    name: string;  
    getSalary:(number)=>number;  
}  
  
class Employee implements IEmployee {  
    empCode: number;  
    name: string;  
  
    constructor(code: number, name: string) {  
        this.empCode = code;  
        this.name = name;  
    }  
  
    getSalary(empCode:number):number {  
        return 20000;  
    }  
}  
  
let emp = new Employee(1, "Steve");
```

* IMPLEMENTING AN INTERFACE

*we create an object called emp of type employee using `let emp = new employee();`. The above class does not include any parameterized constructor so we cannot pass values while creating an object. If the class includes a parameterized constructor, then we can pass the values while creating the object.

Example: Create an Object

```
class Employee {  
    empCode: number;  
    empName: string;  
}  
  
let emp = new Employee();
```

```
class Employee {  
  
    empCode: number;  
    empName: string;  
  
    constructor(empcode: number, name: string ) {  
        this.empCode = empcode;  
        this.name = name;  
    }  
}  
  
let emp = new Employee(100,"Steve");
```

*CREATING AN OBJECT OF CLASS

- * Just like object-oriented languages such as java and C#, typescript classes can be extended to create new classes with inheritance, using the keyword extends.

Example: Inheritance

```
class Person {  
    name: string;  
  
    constructor(name: string) {  
        this.name = name;  
    }  
}  
  
class Employee extends Person {  
    empCode: number;  
  
    constructor(empcode: number, name:string) {  
        super(name);  
        this.empCode = empcode;  
    }  
  
    displayName():void {  
        console.log("Name = " + this.name + ", Employee Code = " + this.empCode);  
    }  
}  
  
let emp = new Employee(100, "Bill");  
emp.displayName(); // Name = Bill, Employee Code = 100
```

* INHERITANCE

* A class can implement single or multiple interfaces

Example: Implement Interface

```
interface IPerson {
    name: string;
    display():void;
}

interface IEmployee {
    empCode: number;
}

class Employee implements IPerson, IEmployee {
    empCode: number;
    name: string;

    constructor(empcode: number, name:string) {
        this.empCode = empcode;
        this.name = name;
    }

    display(): void {
        console.log("Name = " + this.name + ", Employee Code = " + this.empCode);
    }
}

let per:IPerson = new Employee(100, "Bill");
per.display(); // Name = Bill, Employee Code = 100

let emp:IEmployee = new Employee(100, "Bill");
emp.display(); //Compiler Error: Property 'display' does not exist on type 'IEmployee'
```

```

abstract class Person {
  name: string;
  constructor(name: string) {
    this.name = name;
  }

  display(): void {
    console.log(this.name);
  }
  abstract find(string): Person;
}

class Employee extends Person {
  empCode: number;
  constructor(name: string, code: number) {
    super(name); // must call super()
    this.empCode = code;
  }
  find(name: string): Person {
    // execute AJAX request to find an employee from a db
    return new Employee(name, 1);
  }
}

let emp: Person = new Employee("James", 100);
emp.display(); //James
let emp2: Person = emp.find('Steve');

```

- * Typescript allows us to define an abstract class using keyword **abstract**. Abstract classes are mainly for inheritance where other classes may derive from them. We cannot create an instance of an abstract class.
- * An abstract class typically includes one or more abstract methods or property declarations. The class which extends the abstract class must define all the abstract methods.
- * The following abstract class declares one abstract method find and also includes a normal method display.

* TYPESCRIPT - ABSTRACT CLASS

In the last example, person is an abstract class which includes one property and two methods, one of which is declared as abstract. The find() method is an abstract method and so must be defined in the derived class. The employee class derives from the person class and so it must define the find() method as abstract. The employee class must implement all the abstract methods of the person class, otherwise the compiler will show an error.

Note:

The class which implements an abstract class must call super() in the constructor.

The abstract class can also include an abstract property, as shown below.

```
abstract class Person {
  abstract name: string;
  display(): void {
    console.log(this.name);
  }
}

class Employee extends Person {
  name: string;
  empCode: number;
  constructor(name: string, code: number) {
    super(); // must call super()

    this.empCode = code;
    this.name = name;
  }
}

let emp: Person = new Employee("James", 100);
emp.display(); //James
```

- ***public - (default)** allows access to the class member from anywhere
- ***private** - only allows access to the class member from within the class
- ***protected** - allows access to the class member from itself and any classes that inherit it

*Class Members

- *The private access modifier ensures that class members are visible only to that class and are not accessible outside the containing class.

Example: private

```
class Employee {  
    private empCode: number;  
    empName: string;  
}  
  
let emp = new Employee();  
emp.empCode = 123; // Compiler Error  
emp.empName = "Swati";//OK
```

```
class Manager {  
    age: number;  
    constructor(private name: string, age: number, public salary: number)  
    {  
        this.age = age;  
    }  
    show() {  
        console.log("name: " + this.name);  
    }  
}  
  
let man = new Manager("John", 30, 10000);
```

*private

*The protected access modifier is **similar to the private access** modifier, except that protected members can be accessed using their deriving classes

```
class Manager {
  constructor(private name: string, protected age: number, public salary: number) {
    this.age = age;
  }
  show() {
    console.log("name: " + this.name);
  }
}
let man = new Manager("John", 30, 10000);
console.log(typeof man);
class SuperManager extends Manager {
  constructor(name: string, age: number, salary: number) {
    super(name, age, salary);
  }
  show() {
    // console.log("name: " + this.name); error
    console.log("age: " + this.age);
  }
}
```

*protected


```
interface Book {  
  book_id: number;  
  book_name: string;  
}  
interface Author {  
  author_id: number;  
  author_name: string;  
}  
type intersected_type = Book & Author;  
let intersected_type_object1: intersected_type = {  
  book_id: 101,  
  book_name: "Typescript is Awesome",  
  author_id: 202,  
  author_name: "Ahmed ElAshry!",  
};  
console.log("Book Id: " + intersected_type_object1.book_id);  
console.log("Book name: " + intersected_type_object1.book_name);  
console.log("Author Id: " + intersected_type_object1.author_id);  
console.log("Author name: " + intersected_type_object1.author_name);
```

* Intersection Types

* In TypeScript, the generic interface is similar to the interface but it is defined with one or more type parameters. These type parameters allow the interface to be reused with different data types while still enforcing type safety and consistency.

```
// Generic interface
interface IGeneric<T, U> {
  value1: T;
  value2: U;
}
// Define object with a generic interface
let obj: IGeneric<number, string> = {
  value1: 10,
  value2: "Hello",
};
console.log(obj.value2);
```

* Generic Interfaces

In the last example, we have the employee class with two properties- empname and empcode. Since empcode is **read only**, it can be initialized at the time of declaration or in the constructor.

If we try to change the value of empcode after the object has been initialized, the compiler shows the following compilation error:

Error TS2540: cannot assign to empcode' because it is a constant or a read-only property.

An interface can also have read-only member properties.

Example: ReadOnly Interface

```
interface IEmployee {  
    readonly empCode: number;  
    empName: string;  
}  
  
let empObj:IEmployee = {  
    empCode:1,  
    empName:"Steve"  
}  
  
empObj.empCode = 100; // Compiler Error: Cannot change readonly 'empCode'
```

***Read only**

In the same way you can use `read-only<t>` to create a read-only type, as shown below.

Example: ReadOnly Type

```
interface IEmployee {
    empCode: number;
    empName: string;
}

let emp1: Readonly<IEmployee> = {
    empCode: 1,
    empName: "Steve"
}

emp1.empCode = 100; // Compiler Error: Cannot change readonly 'empCode'
emp1.empName = 'Bill'; // Compiler Error: Cannot change readonly 'empName'

let emp2: IEmployee = {
    empCode: 1,
    empName: "Steve"
}

emp2.empCode = 100; // OK
emp2.empName = 'Bill'; // OK
```

*Read only<T>

Export

A module can be defined in a separate .ts file which can contain functions, variables, interfaces and classes. Use the prefix export with all the definitions you want to include in a module and want to access from other modules.

Employee.ts

```
export let age : number = 20;
export class Employee {
  empCode: number;
  empName: string;
  constructor(name: string, code: number) {
    this.empName = name;
    this.empCode = code;
  }
  displayEmployee() {
    console.log ("Employee Code: " + this.empCode + ", Employee Name: " + this.empName );
  }
}
let companyName:string = "XYZ";
```

In the above example, employee.Ts is a module which contains two variables and a class definition. The age variable and the employee class are prefixed with the export keyword, whereas companyname variable is not. Thus, employee.Ts is a module which exports the age variable and the employee class to be used in other modules by importing the employee module using the import keyword. The companyname variable cannot be accessed outside this employee module, as it is not exported.

IMPORT

A module can be used in another module using an import statement.

Let's see different ways of importing a module export.

* Single export from a Module

EmployeeProcessor.ts

```
import { Employee } from "../Employee";  
let empObj = new Employee("Steve Jobs", 1);  
empObj.displayEmployee(); //Output: Employee Code: 1, Employee Name: Steve Jobs
```

* Entire Module into a Variable

EmployeeProcessor.ts

```
import * as Emp from "../Employee"  
console.log(Emp.age); // 20  
  
let empObj = new Emp.Employee("Bill Gates" , 2);  
empObj.displayEmployee(); //Output: Employee Code: 2, Employee Name: Bill Gates
```

- *Triple-slash directives are similar to single-line comments which contain a single XML tag. They are used to provide instructions to the **TypeScript compiler about processing the TypeScript file**. Triple-slash directives start with three forward slashes (`///`) and are mainly placed at the top of the code file.

```
// This file will be referenced in app.ts
Tabnine | Edit | Test | Explain | Document
export function add(a: number, b: number): number {
  return a + b;
}
```

```
/// <reference path="MathFunctions.ts" />
import { add } from "./MathFunctions";
console.log("Addition of 10 and 20 is:", add(10, 20));
```

*Triple-Slash Directives

- * A **namespace** is a way to **logically group related code**. This is inbuilt into TypeScript unlike in JavaScript where variables declarations go into a global scope and if multiple JavaScript files are used within same project there will be possibility of **overwriting** or misconstruing the **same variables**,
- * which will lead to the “global namespace pollution problem” in JavaScript.

```
FileName :IShape.ts
-----
namespace Drawing {
    export interface IShape {
        draw();
    }
}

FileName :Circle.ts
-----
/// <reference path = "IShape.ts" />
namespace Drawing {
    export class Circle implements IShape {
        public draw() {
            console.log("Circle is drawn");
        }
    }
}
```

*Namespaces

- * Decorators in TypeScript are **special kinds of declarations** that can be attached to the **class declaration**, property, accessor, method, and parameters. It is used for the **separate modification of the class** without modifying the original source code.
- * This makes them a powerful tool in the domain of object-oriented programming, allowing you to write cleaner, more organized code that adheres to the DRY (**Don't Repeat Yourself**) principle.

```
function LogClass(constructor: Function) {  
    console.log(`Class created: ${constructor as any}.name`);  
}  
  
@LogClass  
class MyClass {  
    constructor() {  
        console.log("Instance created!");  
    }  
}  
  
let x = new MyClass();
```

*Decorators